

Elevate Fitness App — Complete Backend Engine Blueprint

Section 1: Global Architectural Blueprint & Gateway Configuration

1.1 Base Configuration

- **Base URL Matrix:** `/api/v1`
- **Default Content-Type:** `application/json`
- **Authorization Protocol:** JWT Bearer Token passed via the HTTP Header:
`Authorization: Bearer {token}`

1.2 Unified Standard Response Envelope

All microservices must map outbound data to this consistent structural schema:

```
{
  "isSuccess": true,
  "message": "Operation completed successfully.",
  "data": {},
  "errors": [],
  "statusCode": 200,
  "timestamp": "2026-06-17T16:38:00Z"
}
```

1.3 API Gateway Routing & Rate Limiting Strategy

The API Gateway serves as the edge proxy checking user tiers against cached premium validation configurations. The gatekeeper rules enforce hard rate limits using standard headers (`X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`):

User Classification	Rate Limit Rule	Access Tier Capability
Anonymous	50 requests / hour	Registration, login, password recovery flows only.
Authenticated (Free)	500 requests / hour	Access to base features; soft-capped system gates.

User Classification	Rate Limit Rule	Access Tier Capability
Premium	2000 requests / hour	Full unthrottled access; advanced metrics, unrestricted coach conversations.

Section 2: Microservice-by-Microservice Deep Dive (APIs & Database Schemas)

1. Authentication Service

- **Data Ownership:** Holds explicit, exclusive boundaries for user access credentials, token rotation states, and security auditing logs. It does not contain personal profiles or fitness parameters.

Database Schema (Relational Model)

Table: Users

- `Id` (int, PK, Identity): Unique security identifier.
- `Email` (string, Unique Index, `VarChar(255)`): User registration email.
- `PasswordHash` (string, `VarChar(512)`): Cryptographically hashed/salted password.
- `IsLockedOut` (bool): Security flag for consecutive failures.
- `LockedUntil` (DateTime?, Nullable): Expiration timestamp of the lockout penalty.
- `CreatedAt` (DateTime): Record initialization timestamp.

Table: OtpCodes

- `Id` (int, PK, Identity): Primary Identifier.
- `Email` (string, Index, `VarChar(255)`): Associated account destination.
- `Code` (string, `VarChar(6)`): Hashed 6-digit challenge value.
- `ExpiresAt` (DateTime): Expiration boundary (Exactly 10 minutes from creation).
- `IsUsed` (bool): Consumed flag preventing replay attacks.

Table: RefreshTokens

- `Id` (int, PK, Identity): Primary Identifier.
- `UserId` (int, FK -> Users.Id): Target relational account.
- `Token` (string, Unique Index, `VarChar(512)`): Cryptographic token value.
- `ExpiresAt` (DateTime): Maximum active validation limit.

- `CreatedAt` (DateTime): Creation log.
- `RevokedAt` (DateTime?, Nullable): Cancellation timestamp.

Table: LoginAttempts

- `Id` (int, PK, Identity): Audit row identifier.
- `Email` (string, VarChar(255)): Entered address.
- `AttemptedAt` (DateTime): Precise tracking time.
- `IsSuccess` (bool): Indicator of successful password execution.
- `IpAddress` (string, VarChar(45)): Client IP tracker.

API Endpoints Directory

- `POST /api/v1/auth/register` (Anonymous)
- *Payload*: `{"firstName": "Ahmed", "lastName": "Mohamed", "email": "ahmed@example.com", "password": "Ahmed@123", "phoneNumber": "+201234567890"}`
- *Validation*: Name must be between 2-50 characters. Egyptian phone match required. Strong password evaluation enforced.
- *Response (201)*: Returns `userId` and sets `requiresProfileCompletion: true`.
- `POST /api/v1/auth/complete-profile` (Authenticated)
- *Payload*: Triggers upstream internal events to notify downstream profile records that registration setup states are closed.
- *Response (200)*: `{"isSuccess": true, "message": "Profile lifecycle initiated."}`
- `POST /api/v1/auth/login` (Anonymous)
- *Payload*: `{"email": "ahmed@example.com", "password": "Ahmed@123"}`
- *Business Rules*: Evaluates `LoginAttempts`. If failures hit 5 within 15 minutes, marks `IsLockedOut = true`, updates `LockedUntil`, and yields a 423 Locked error returning `AUTH_ACCOUNT_LOCKED`.
- *Response (200)*: Returns standard access structures containing `token`, `refreshToken`, and cached boolean flags (`profileCompleted`, `isPremium`).
- `POST /api/v1/auth/forgot-password` (Anonymous)
- *Payload*: `{"email": "ahmed@example.com"}`
- *Business Rules*: Generates a 6-digit code to `OtpCodes`. Enforces a 30-second resend limit.
- *Response (200)*: `{"data": {"email": "ahmed@example.com", "otpExpiresIn": 600, "canResendIn": 30}}`
- `POST /api/v1/auth/verify-otp` (Anonymous)
- *Payload*: `{"email": "ahmed@example.com", "otp": "123456"}`
- *Response (200)*: Returns short-lived `resetToken`.
- `POST /api/v1/auth/reset-password` (Anonymous)

- **Payload:** {"resetToken": "xyz", "newPassword": "New@123", "confirmPassword": "New@123"}
 - **Response (200):** {"data": {"passwordChanged": true}}
 - **POST /api/v1/auth/refresh-token** (Anonymous)
 - **Payload:** {"refreshToken": "token_str"}
 - **Response (200):** Supplies refreshed cryptographic token combinations.
 - **POST /api/v1/auth/logout** (Authenticated)
 - **Response (200):** Revokes current active token access status immediately.
-

2. User Profile Service

- **Data Ownership:** Contains personal contact structures, regional localization settings, and layout preference flags. It subscribes to the billing pipeline to retain a cached replica of `IsPremiumCached` for data response lookups.

Database Schema

Table: UserProfiles

- `UserId` (int, PK): Matches identity database reference safely.
- `FirstName` (string, VarChar(50)) / `LastName` (string, VarChar(50))
- `Email` (string, VarChar(255))
- `PhoneNumber` (string, VarChar(20))
- `ProfilePictureUrl` (string, VarChar(500), Nullable)
- `IsPremiumCached` (bool): Local cache synced from Subscription Microservice events.
- `MemberSince` (DateTime): Application profile creation date.

Table: UserPreferences

- `UserId` (int, PK, FK -> UserProfiles.UserId)
- `Language` (string, VarChar(10), Default: 'en')
- `Theme` (string, VarChar(15), Default: 'light')
- `WeightUnit` (string, VarChar(5), Default: 'kg')
- `HeightUnit` (string, VarChar(5), Default: 'cm')
- `DistanceUnit` (string, VarChar(5), Default: 'km')

Table: NotificationSettings

- `UserId` (int, PK, FK -> UserProfiles.UserId)

- `WorkoutReminders / MealReminders / AchievementAlerts / WeeklyReports / EmailNotifications / PushNotifications` (bool, Default: true)

Table: `PrivacySettings`

- `UserId` (int, PK, FK -> `UserProfiles.UserId`)
- `ProfileVisibility` (string, `VarChar(20)`, Default: 'private')
- `ShowProgressToFriends` (bool, Default: false)
- `AllowDataSharing` (bool, Default: false)

API Endpoints Directory

- GET `/api/v1/profile` (Authenticated)
- *Response (200)*: Composes personal context fields from `UserProfiles` and merges data aggregations representing high-level metric snapshots.
- PUT `/api/v1/profile` (Authenticated)
- *Payload*: `{"firstName": "Ahmed", "lastName": "Ali", "email": "ahmed.ali@example.com", "phoneNumber": "+201234567890"}`
- POST `/api/v1/profile/picture` (Authenticated)
- *Payload*: Multi-part Form Data (`profilePicture` binary file; max size limit 5MB, format limited to JPG/PNG).
- PUT `/api/v1/profile/change-password` (Authenticated)
- *Payload*: `{"currentPassword": "Old", "newPassword": "New", "confirmPassword": "New"}`
- GET `/api/v1/settings` (Authenticated)
- *Response (200)*: Returns a grouped JSON view of `UserPreferences` , `NotificationSettings` , and `PrivacySettings` .
- PUT `/api/v1/settings` (Authenticated)
- *Payload*: Accepts full or partial JSON variants updating specific key configurations safely.

3. Fitness Calculation Engine (FCE)

- **Data Ownership**: Owns physical biometric logs, mathematically derives metabolic constraints (BMR, TDEE, Calorie Targets), manages rule-based intensity levels, and tracks the historical timeline of structural plan changes.

Database Schema

Table: `UserFitnessStats`

- `Id` (int, PK, Identity): Metrics snapshot entry ID.
- `UserId` (int, Index): Target owner indicator.
- `Weight` (double) / `Height` (double) / `Age` (int)
- `Gender` (string, VarChar(10)) — Restricted via validation rules to: `Male` , `Female` .
- `Goal` (string, VarChar(50)) — Configured choices: `Lose Weight` , `Get Fitter` , `Gain Weight` , `Gain More Flexible` , `Learn the Basic` .
- `ActivityLevel` (string, VarChar(30)) — Allowed levels: `Rookie` , `Beginner` , `Intermediate` , `Advance` , `TrueBeast` .
- `RecordedAt` (DateTime): Exact entry log timestamp.

Table: CalculatedMetrics

- `Id` (int, PK, Identity): Primary Identifier.
- `UserId` (int, Unique Index): Current active calculated output container per user.
- `Bmr` / `Tdee` / `CalorieTarget` (double): Computed scientific properties.
- `Status` (string, VarChar(20)): Assigned tier boundary (`Weak` , `Normal` , `Hard`).
- `CalculatedAt` (DateTime): Generation timestamp.

Table: FitnessPlanConfig

- `PlanId` (string, PK, VarChar(50)): Structural identifier matching Workout Service mapping targets.
- `PlanName` (string, VarChar(100))
- `Description` (string, VarChar(500))
- `Goal` (string, VarChar(50))
- `Status` (string, VarChar(20))
- `MinCalorie` / `MaxCalorie` (double): Binding boundary checks.
- `EstimatedDuration` (string) / `WorkoutsPerWeek` (int) / `ProgramType` (string)

Table: UserAssignedPlans

- `Id` (int, PK, Identity): Unique relational entry assignment.
- `UserId` (int, Index): Target context user identifier.
- `PlanId` (string, FK -> FitnessPlanConfig.PlanId): Assigned plan mapping key.
- `AssignedAt` (DateTime): Strategy initialization date.
- `IsActive` (bool): Identifies if this is the active track.

Table: UserPlanHistory

- `Id` (int, PK, Identity): Historical log indicator.
- `UserId` (int, Index): Target owner indicator.

- `PlanId` (string): Record of past plan state target identifier.
- `AssignedAt` (DateTime): Plan activation date.
- `EndedAt` (DateTime?): Reassignment timestamp.
- `ReasonForChange` (string, VarChar(255)): Reason for calculation updates (e.g., "weight_update").

Mathematical Engine Formulas

- **BMR Calculations:**
 - *Male:* $BMR = (10 \times \text{weight}_{\text{kg}}) + (6.25 \times \text{height}_{\text{cm}}) - (5 \times \text{age}_{\text{years}}) + 5$
 - *Female:* $BMR = (10 \times \text{weight}_{\text{kg}}) + (6.25 \times \text{height}_{\text{cm}}) - (5 \times \text{age}_{\text{years}}) - 161$
- **TDEE Calculation:** $TDEE = BMR \times \text{Activity Factor}$
- **Activity Multipliers:** Rookie = 1.2, Beginner = 1.375, Intermediate = 1.55, Advance = 1.725, TrueBeast = 1.9
- **Calorie Target Assignment Matrix:**
 - Lose Weight = $TDEE - 500$
 - Get Fitter = $TDEE$
 - Gain Weight = $TDEE + 300$
 - Gain More Flexible = $TDEE + 150$
 - Learn the Basic = $TDEE$
- **System Status Classification:**
 - ≤ 1800 calories: Weak
 - 1801 to 2500 calories: Normal
 - > 2500 calories: Hard

API Endpoints Directory

- POST `/api/v1/fitness/weight-goal-activity` (Authenticated)
- **Payload:** `{"userId": "usr_123456", "weight": 75.5, "height": 175, "age": 25, "gender": "Male", "goal": "Lose Weight", "activityLevel": "Intermediate"}`
- **Validation Constraints:** Age (16-100), Weight (40-200kg), Height (140-220cm). Strict option boundaries checked on Goal and Activity strings.
- POST `/api/v1/fitness/calculate` (Authenticated)
- **Payload:** `{"userId": "usr_123456"}`
- **Business Logic:** Parses current active `UserFitnessStats`, executes structural BMR/TDEE math formulas, writes to `CalculatedMetrics`, and outputs results.
- POST `/api/v1/fitness/assign-plan` (Authenticated)
- **Payload:** `{"userId": "usr_123456"}`

- **Business Logic:** Matches calculated target metrics against `FitnessPlanConfig` thresholds. Writes a new record to `UserAssignedPlans` and appends entries into `UserPlanHistory` automatically.
 - `GET /api/v1/fitness/metrics/{userId}` (Authenticated)
 - `GET /api/v1/fitness/stats/{userId}` (Authenticated)
 - `PUT /api/v1/fitness/recalculate/{userId}` (Authenticated)
 - **Payload (Optional):** `{"reason": "weight_update", "newWeight": 73.0, "triggeredBy": "progress_service"}`
 - **Response (200):** Returns comparative change nodes containing `previousMetrics`, `newMetrics`, and updates `UserPlanHistory`.
 - `GET /api/v1/fitness/plan-configs` (Authenticated)
 - `GET /api/v1/fitness/plans/{planId}` (Authenticated)
 - **Response (200):** Resolves specific deep structure rules from the configuration table matching requested indices.
-

4. Workout Service 💪

- **Data Ownership:** Core authority governing exercise metadata templates, workout step configurations, default metrics setups, and the runtime state mapping table for active user exercise tracks.

Database Schema

Table: `WorkoutPlans`

- `PlanId` (string, PK, `VarChar(50)`): Structural key pairing directly with FCE system rules.
- `Name` (string, `VarChar(100)`)
- `Description` (string, `VarChar(500)`)
- `Goal` (string, `VarChar(50)`)
- `Status` (string, `VarChar(20)`)
- `Difficulty` (string, `VarChar(20)`) — Enforced scales: `beginner`, `intermediate`, `advanced`.

Table: `Workouts`

- `WorkoutId` (int, PK, Identity): Primary Identifier.
- `PlanId` (string, FK -> `WorkoutPlans.PlanId`): Connected group structural plan.
- `Name` (string, `VarChar(100)`)

- **Category** (string, VarChar(50), Index): full-body , chest , arms , shoulders , back , legs , stomach .
- **DurationInMinutes** (int)
- **Difficulty** (string, VarChar(20))

Table: Exercises

- **ExerciseId** (int, PK, Identity): Standard exercise library reference.
- **Name** (string, VarChar(100))
- **TargetMuscles** (string, VarChar(255)): Comma-separated list of targeted muscles.
- **Equipment** (string, VarChar(150))
- **Difficulty** (string, VarChar(20))
- **Description** (string, VarChar(1000))
- **VideoUrl** (string, VarChar(500), Nullable)

Table: WorkoutExercises

- **Id** (int, PK, Identity): Primary execution index key.
- **WorkoutId** (int, FK -> Workouts.WorkoutId): Parent layout.
- **ExerciseId** (int, FK -> Exercises.ExerciseId): Linked baseline move.
- **SetsDefault** (int)
- **RepsDefault** (int)
- **RestTimeInSeconds** (int)
- **OrderIndex** (int): Sorted order positioning sequence.

Table: WorkoutSessions

- **SessionId** (string, PK, VarChar(100)): Globally unique tracking token.
- **UserId** (int, Index): User processing thread.
- **WorkoutId** (int, FK -> Workouts.WorkoutId): Selected core training catalog.
- **StartedAt** (DateTime): Operational state initialization boundary.
- **Status** (string, VarChar(20)): Tracking system status flag (Active , Completed , Abandoned).

API Endpoints Directory

- **GET /api/v1/workouts** (Authenticated)
- **Query Options:** page , pageSize , category , difficulty , duration , search .
- **GET /api/v1/workouts/{id}** (Authenticated)
- **Response (200):** Yields full workout mapping matrices with matching WorkoutExercises structures and variation rules.

- GET /api/v1/workouts/by-plan/{planId} (Authenticated)
 - GET /api/v1/workouts/category/{categoryName} (Authenticated)
 - POST /api/v1/workouts/{id}/start (Authenticated)
 - *Payload*: {"difficulty": "Intermediate", "plannedDuration": 45}
 - *Business Logic*: Initializes a new session tracking row in `WorkoutSessions` under state "Active" .
 - *Response (201)*: {"data": {"sessionId": "sess_uuid_999", "orderedExercises": [...]}}
 - GET /api/v1/exercises (Authenticated)
 - GET /api/v1/exercises/{id} (Authenticated)
 - GET /api/v1/workout-plans (Authenticated)
 - GET /api/v1/workout-plans/{planId} (Authenticated)
-

5. Nutrition Service 🥗

- **Data Ownership**: Manages structural food catalogs, dietary ingredient databases, recipe preparations, and composite calorie grouping frameworks matching specific metric boundaries.

Database Schema

Table: Meals

- `MealId` (int, PK, Identity): Unique menu sequence.
- `Name` (string, `VarChar(150)`)
- `Type` (string, `VarChar(20)`): Classified type (`Breakfast` , `Lunch` , `Dinner` , `Snack`).
- `Calories` / `Protein` / `Carbs` / `Fats` (double): Core nutritional components.
- `IngredientsJson` (string, Text Data): Structured text containing exact ingredient ratios.
- `InstructionsJson` (string, Text Data): Ordered text list detailing specific cooking instructions.
- `VariationsJson` (string, Text Data, Nullable): Standard modifications for beginner or advanced preparation tracks.
- `AllergensJson` (string, `VarChar(500)`): Array strings denoting allergens (e.g., [`"Nuts"` , `"Dairy"`]).
- `TagsJson` (string, `VarChar(255)`): Meta classifiers (`"High Protein"` , `"Keto"`).

Table: MealPlans

- `MealPlanId` (int, PK, Identity): Blueprint index.

- Name (string, VarChar(100))
- Description (string, VarChar(500))
- TargetCalorieRangeMin / TargetCalorieRangeMax (double): Verification range bindings.

Table: MealPlanItems

- Id (int, PK, Identity): Relational assignment key.
- MealPlanId (int, FK -> MealPlans.MealPlanId): Connected template tracker.
- MealId (int, FK -> Meals.MealId): Targets reference mapping rows.
- DayOfWeek (string, VarChar(15)): Schedule placement markers.
- MealTime (string, VarChar(20)): Meal time slot classification.

API Endpoints Directory

- GET /api/v1/nutrition/recommendations (Authenticated)
- *Parameters*: mealType , page , pageSize , maxCalories , minProtein .
- *Response (200)*: Merges custom user daily targets alongside matching meal option configurations.
- GET /api/v1/nutrition/recommendations/{userId} (Authenticated)
- *Business Logic*: Queries internal calorie target levels calculated inside FCE to extract filtered recommendations matching precise target ranges automatically.
- GET /api/v1/nutrition/meals/{id} (Authenticated)
- GET /api/v1/nutrition/meal-plans (Authenticated)
- GET /api/v1/nutrition/meal-plans/by-calories (Authenticated)
- *Query*: ?calories=2200 — Filters plans where target calorie parameters fit cleanly inside system ranges.

6. Smart Coach Service 🤖

- **Data Ownership**: Coordinates real-time user-to-AI dialogue text histories, system prompt tracking states, and temporary cache storage records for user landing feed displays.

Database Schema

Table: ChatSessions

- SessionId (string, PK, VarChar(100)): Tracking session key.
- UserId (int, Index): Associated user index reference.
- CreatedAt (DateTime): Start timestamp.

- `Title` (string, `VarChar(150)`, Default: 'New Conversation')

Table: `ChatMessages`

- `MessageId` (int, PK, Identity): Row identifier.
- `SessionId` (string, FK -> `ChatSessions.SessionId`): Structural group index tracker.
- `Sender` (string, `VarChar(10)`): Actor state identifier (`User` , `AI`).
- `Content` (string, Text Data): Output chat copy block.
- `Timestamp` (`DateTime`): Creation log.

Table: `RecommendationCache`

- `UserId` (int, PK): System identifier.
- `UserContextJson` (string, Text Data): Stores snapshots containing active weight properties, calculation goals, target limits, and current execution plans.
- `HomeFeedDataJson` (string, Text Data): Full aggregated payload container snapshot caching user dashboard metrics.
- `CachedAt` (`DateTime`): Generation log.
- `ExpiresAt` (`DateTime`): Cache expiration tracking marker.

API Endpoints Directory

- `POST /api/v1/smart-coach/chat` (Authenticated)
- *Payload*: `{"message": "How do I optimize my target metrics?", "sessionId": "sess_111"}`
- *Business Logic*: Pulls user state fields internally via FCE/Progress modules, feeds parameters safely inside AI prompt engines, caches the result state, and stores values in the chat message log tables.
- `GET /api/v1/smart-coach/history` (Authenticated)
- `GET /api/v1/home` (Authenticated)
- *Business Logic*: Serves as an edge aggregator endpoint. Checks if `RecommendationCache` holds valid, non-expired data (`DateTime.UtcNow < ExpiresAt`). If invalid, fetches data asynchronously across downstream microservices (Profiles, FCE targets, Workout lists, Nutrition plates, Progress tracks), updates the cache row, and outputs the aggregated result block.

7. Progress Tracking Service

- **Data Ownership:** Tracks user activity completion data, body biometric progress tracking scales, current streak counters, statistics aggregations, and earned rewards markers.

Database Schema

Table: WorkoutLogs

- **Id** (int, PK, Identity): Execution entry database index.
- **UserId** (int, Index): Target tracking actor identifier.
- **WorkoutId** (int): Target reference tracking reference mapping key.
- **SessionId** (string, Index): Active token tracking matching the initial context tracking state.
- **DurationInMinutes** (int)
- **CaloriesBurned** (int)
- **Rating** (int) — Verified boundary conditions check: Scale (1 to 5).
- **Notes** (string, VarChar(1000), Nullable)
- **CompletedAt** (DateTime): Processing end log.

Table: WorkoutLogExercises

- **Id** (int, PK, Identity): Execution detail row ID.
- **WorkoutLogId** (int, FK -> WorkoutLogs.Id): Relational parent record index mapping constraint.
- **ExerciseId** (int): Maps target ID reference cleanly.
- **SetsCompleted** (int)
- **RepsCompleted** (int)
- **WeightUsed** (double, Default: 0)
- **Completed** (bool): Identifies success completion loop states.

Table: WeightHistory

- **Id** (int, PK, Identity): Tracking row log sequence ID.
- **UserId** (int, Index): Target mapping tracking element.
- **Weight** (double)
- **Date** (DateTime): Day recording baseline marker.
- **Notes** (string, VarChar(500), Nullable)

Table: BodyMeasurements

- **Id** (int, PK, Identity): Measurement snapshot entry ID.
- **UserId** (int, Index): Target system user tracking link.
- **Neck / Chest / Biceps / Waist / Hips / Thighs** (double, Nullable)

- RecordedAt (DateTime): Calendar tracking event.

Table: Achievements

- Id (int, PK, Identity): Core master tier tracking index.
- Name (string, VarChar(100))
- Description (string, VarChar(255))
- IconUrl (string, VarChar(255))

Table: UserAchievements

- Id (int, PK, Identity): Linked achievement mapping reference key.
- UserId (int, Index): Target profile indexing criteria.
- AchievementId (int, FK -> Achievements.Id): Connects master award definition.
- EarnedAt (DateTime): Goal realization event tracker.

Table: Streaks

- UserId (int, PK): System identifier matching context profile states.
- CurrentStreak (int, Default: 0)
- LongestStreak (int, Default: 0)
- LastWorkoutDate (DateTime?, Nullable): Used to check daily calculation intervals.

Table: UserStatistics

- UserId (int, PK): Base entity record anchor.
- TotalWorkouts (int, Default: 0)
- TotalCaloriesBurned (int, Default: 0)
- TotalWeightLost (double, Default: 0)
- UpdatedAt (DateTime): Modification state timestamp.

API Endpoints Directory

- GET /api/v1/progress (Authenticated)
- *Parameters:* period (week, month, year, all), startDate, endDate.
- GET /api/v1/progress/{userId} (Authenticated)
- POST /api/v1/progress/workouts (Authenticated)
- *Payload:*

```
{
  "workoutId": 1,
  "sessionId": "session_xyz789",
```

```

    "completedAt": "2026-06-17T11:15:00Z",
    "duration": 45,
    "caloriesBurned": 350,
    "difficulty": "Intermediate",
    "notes": "Pushed harder on last set",
    "rating": 5,
    "exercisesCompleted": [
      { "exerciseId": 1, "sets": 3, "reps": 15, "weightUsed": 60.0, "completed":
true }
    ]
  }
}

```

- **Business Logic:** Parses `sessionId` and verifies its authenticity. Commits data lines to `WorkoutLogs` and appends entries inside `WorkoutLogExercises`. Updates `Streaks` and `UserStatistics` values, evaluates achievement eligibility metrics, and emits message broker alerts upon completing goals.
- POST `/api/v1/progress/weight` (Authenticated)
- **Payload:** `{"weight": 72.5, "date": "2026-06-17", "notes": "Morning check"}`
- **Business Logic:** Appends entries to `WeightHistory`. Emits an asynchronous broker command (`weight_updated`) targeting FCE to recalculate target biometrics automatically. Returns contextual calculations (e.g., progress percentage to goal).
- GET `/api/v1/progress/weight-history/{userId}` (Authenticated)
- GET `/api/v1/progress/achievements` (Authenticated)
- GET `/api/v1/progress/stats/{userId}` (Authenticated)

8. Subscription & Billing Service

- **Data Ownership:** This microservice acts as the explicit source of truth for user financial account states, access authorization boundaries, tier levels, and billing metrics.

Database Schema

Table: UserSubscriptions

- `Id` (int, PK, Identity): Standard tier indicator ID.
- `UserId` (int, Unique Index): Distinct customer target profile.
- `Tier` (string, VarChar(20), Default: 'Free'): Tier values enforced (`Free` , `Premium`).
- `Status` (string, VarChar(20), Default: 'Active'): Evaluation track states (`Active` , `Expired` , `Cancelled`).

- `StartsAt` (DateTime): Contract state initialization timestamp.
- `ExpiresAt` (DateTime): Plan termination processing date.
- `UpdatedAt` (DateTime): Record tracking index entry modification event.

Table: BillingLogs

- `Id` (int, PK, Identity): Audit billing reference row ID.
- `UserId` (int, Index): Context profile owner indicator tracking link.
- `ReferenceId` (string, VarChar(100)): External dummy registration receipt tracker string.
- `Amount` (decimal(18,2)): Financial evaluation data format tracker.
- `Currency` (string, VarChar(3), Default: 'EGP')
- `PaymentStatus` (string, VarChar(20)): Log summary state indicator values (`Success` , `Failed`).
- `Timestamp` (DateTime): Transaction execution record marker.

Soft-Capped System Business Rules

- **Free Tier Boundaries:** Enforces standard operations processing loops. Limits AI conversation tracks to 5 messages every 24 hours via caching checks. Restricts advanced calculation outputs and blocks custom training logic configurations.
- **Premium Level Activations:** Upon successful row creation or mock transaction validation updates, the engine publishes an event (`subscription_upgraded`) across message brokers. Downstream, the Profile Service catches this event to update `IsPremiumCached = true` . The API Gateway also updates its local distributed caching models to dynamically lift rate limit restrictions to 2000 requests per hour.

API Endpoints Directory

- `GET /api/v1/subscription/status/{userId}` (Authenticated)
 - *Response (200):* Returns active customer tiers (`Free` / `Premium`), validation expirations, and rule conditions.
 - `POST /api/v1/subscription/upgrade` (Authenticated)
 - *Payload:* `{"userId": "usr_123456", "planTier": "Premium", "durationMonths": 1}`
 - *Business Logic:* Simulates billing authorization updates. Inserts a record to `BillingLogs` , initializes or updates `UserSubscriptions` to `"Premium"` , sets the expiration date, and fires synchronization events across system networks.
 - `POST /api/v1/subscription/cancel` (Authenticated)
 - *Response (200):* Safely transitions system flags to cancel auto-renewal setups without breaking active timelines.
-

9. Notification Service

- **Data Ownership:** Functions as a simple system tracking block handling routing mechanics for tracking indicators and storing inside-app alerts histories.

Database Schema

Table: InAppNotifications

- `Id` (int, PK, Identity): System reference key.
- `UserId` (int, Index): Target recipient profile account ID tracking link.
- `Title` (string, VarChar(150)) / `Message` (string, VarChar(500))
- `Type` (string, VarChar(30)): Category classifications (`AchievementAlert` , `StreakWarning` , `SystemUpdate`).
- `IsRead` (bool, Default: false): Interaction state tracker.
- `SentAt` (DateTime): Outbound distribution log time.

API Endpoints Directory

- `GET /api/v1/notifications` (Authenticated)
 - *Response (200):* Yields active user unread notification logs from `InAppNotifications` .
 - `PUT /api/v1/notifications/{id}/read` (Authenticated)
 - *Business Logic:* Toggles the selected record status to `IsRead = true` instantly.
-

Section 3: Comprehensive Error Code Registry & HTTP Mappings

When a request falls out of structural bounds, handlers must catch the pipeline thread and format the error using the standardized system structure containing precise application codes:

HTTP Core Map Matrix

- `200 OK` / `201 Created` / `204 No Content`
- `400 Bad Request` (Payload errors, schema validation constraints bypassed)
- `401 Unauthorized` (Token signature failures, missing security components)
- `403 Forbidden` (Premium level required, incomplete configurations blocks)
- `404 Not Found` (Target identifiers missing inside entity tracking blocks)
- `409 Conflict` (Duplicate record registration constraints violated)
- `423 Locked` (Lockout state limits triggered due to access violations)

- 429 Too Many Requests (Rate tracking thresholds exceeded)
- 500 Internal Error / 503 Service Unavailable

Explicit System Application Error Codes

Authentication & Validation Errors (AUTH_* / VAL_*)

- AUTH_INVALID_CREDENTIALS (401): Password or email validation parameters rejected.
- AUTH_ACCOUNT_LOCKED (423): User locked out after 5 invalid attempts. Holds an active 15-minute verification freeze.
- AUTH_TOKEN_EXPIRED (401): Request authorization context dropped because the JWT timeline expired.
- AUTH_TOKEN_INVALID (401): Signature check failure on parsing incoming structures.
- AUTH_EMAIL_EXISTS (409): Target registration account parameters match a preexisting user record row.
- VAL_REQUIRED_FIELD (400): Inbound JSON payload is missing vital structural context parameters.
- VAL_INVALID_AGE / VAL_INVALID_WEIGHT / VAL_INVALID_HEIGHT (400): User stats parameter values violate physical model constraints.

Resource & Business Processing Core Errors (RES_* / FCE_* / PERM_*)

- RES_NOT_FOUND (404): Generic baseline entity row extraction failure.
- RES_SESSION_NOT_FOUND (404): System reference code matching SessionId does not exist inside active WorkoutSessions .
- FCE_METRICS_NOT_CALCULATED (400): Dependent system process dropped because target user calculation indexes do not exist.
- PERM_PREMIUM_REQUIRED (403): Operation dropped by the edge proxy because user verification flags require active Premium tiers.
- RATE_LIMIT_EXCEEDED (429): Operations traffic tracking layer exceeds baseline configuration limits.

Section 4: Architecture Communication & Interface Layout

Inter-Service Communication Layout Matrix

[Synchronous REST/HTTP Calls]

API Gateway -----> Auth Service (Token Signing / Lockout Verification)

Smart Coach -----> FCE Service (Read Active Calculations Matrix)
Smart Coach -----> Progress Tracking (Read Active Streak Values)
Nutrition Service -----> FCE Service (Extract Live Target Calorie Metrics)

[Asynchronous Event-Driven Pub/Sub via Message Broker]

Progress Service -----> (Event: weight_updated) -----> FCE Service
(Trigger Recalculate)

Subscription Service -----> (Event: subscription_upgraded) -> User Profile
(Sync Cache Flag)

Progress Service -----> (Event: achievement_earned) -----> Notification
Service Stub